DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY, ANNA UNIVERSITY, CHENNAI – 600 044

FINAL REVIEW REPORT

AUTONOMOUS AI-AUGMENTED CI/CD PIPELINE WITH ADAPTIVE TRUST-TIERING AND CHAOS ENGINEERING

Team 18

Praveen Kumar C · Jaiprakash S · Dharshini T

B.Tech Artificial Intelligence and Data Science

Project guide: Dr. P. Anandha Kumar

Department of Information Technology

[Review date]

1. Executive Summary

When a CI/CD pipeline run fails, someone has to decide what to do: retry it, escalate it, take a machine out of service, or stop the merge. Our project builds an agent that makes that decision from the run's distributed trace, and, more importantly, decides **how much it is allowed to do on its own** based on how often it has been right.

This report covers the final phase. Since Review 2 we added the release-gate layer (trace adapter and drift guard, canary comparison, vulnerability-reachability check, chaos experiments, actuator and CI hook), a runnable live system with injectable faults, an interactive dashboard, and a comparison with published model architectures re-implemented on our own data and split.

Table 1: Headline results (all measured; details and caveats in the sections that follow)

Item	Result	Read it with this caveat
Triage model, HDFS test set (10,513 traces, 57 unseen scenarios)	macro-F1 0.7464, accuracy 0.7690	0.6829 excluding REAL_BUG, whose 1.000 is a trace-length effect
Comparison with 4 published architectures, same data and recipe	ours 0.7464 vs 0.6115, 0.6549, 0.4446, 0.6059, 0.7672	Re-implementations, single run each; not the papers' own numbers
Trust engine on the test stream	T0 → T3 on evidence; demoted after injected faults	Promotion needs a Wilson lower bound; demotion is immediate
Live closed loop (390 requests)	362 of 390 correct (92.8%), 2 unsafe actions	Toy system; the silent bug is invisible by design
Vulnerability gate	27 flagged, 0 reachable	Static import analysis only
Automated tests	183 passed, 12 skipped	Skips: no opa executable; one legacy marker

In plain words. The model is a solid but not spectacular classifier. What makes the project distinctive is the safety layer around it: the agent earns autonomy from evidence, loses it the moment it merges a real defect, refuses to judge traces it was not trained for, and every limit of the system is measured and stated.

2. What Changed Since the Earlier Reviews

Table 2: Progress across the three reviews

Review	What was shown	Main limitation at that point
Review 1	Phase 1 prototype: an LSTM reporting 88.18% accuracy; project proposal	The 88.18% was a one-column rule: rows were event-count vectors fed to an embedding as token ids
Review 2	Rebuilt data foundation (clock-aligned sequences, scenario split), attention BiLSTM, trust tiers, root-cause ranking, demo	Everything ran offline on one data set; no live system; no release-gate signals
Final (this report)	Adapter and drift guard, canary, security gate, chaos harness, actuator and CI hook, live minishop system, dashboard, published-model comparison	Workflows, Grafana/Prometheus stack and real-infrastructure chaos were written but never run

3. System Overview

The system is five layers: trace sources, an adapter that normalises them and checks the model is entitled to judge them, the decision components (triage model, root-cause localiser, trust engine), the release gate that adds security and canary signals, and an output layer that executes decisions in dry-run mode and feeds the true outcome back.

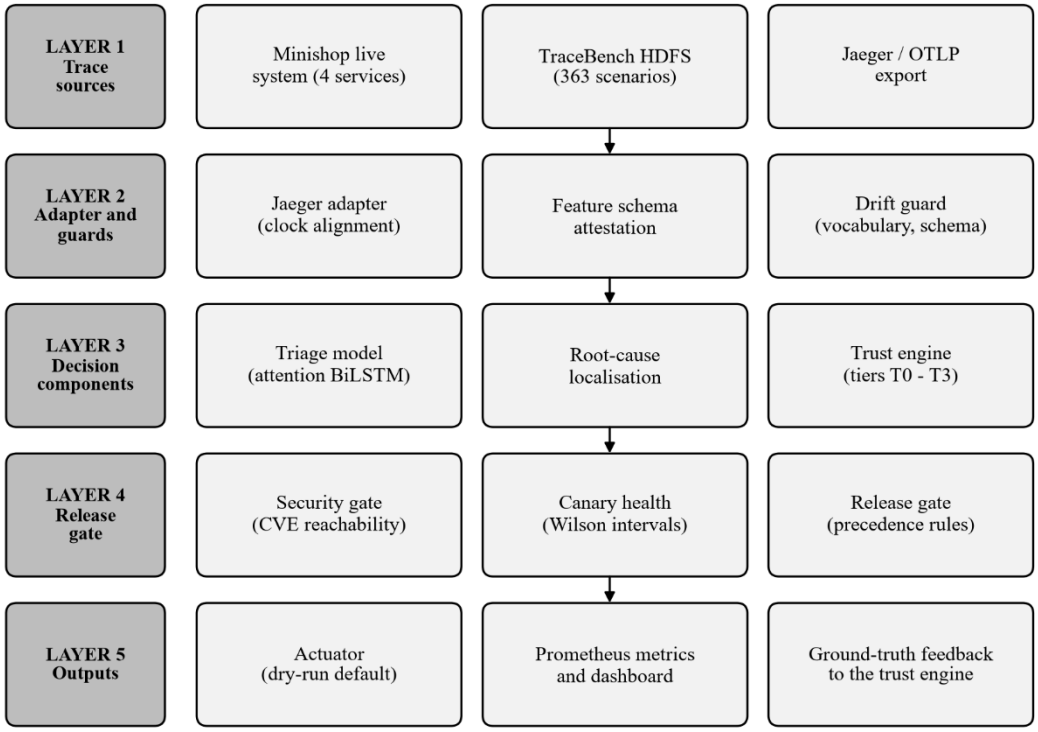


Figure 1: Architecture of the proposed system

The five classes the model predicts, and what each one makes the pipeline do:

Table 3: Classes, actions and the minimum trust tier needed to execute them

Class	Meaning	Action	Min. tier
PASS	No failure signature	ALLOW_MERGE	T2
INFRA_FLAKE	Transient network or process fault	AUTO_RETRY_PIPELINE (budget 2)	T2
DATA_FAULT	Corruption or loss; deterministic	ESCALATE_TO_INFRA (never retry)	T1
ENV_FAULT	Host-level failure	QUARANTINE_NODE	T3
REAL_BUG	Genuine defect signature	BLOCK_MERGE_ALERT_DEV	T1

3.1 How to run it

- **Dashboard (the demo):** python scripts/serve_ui.py, then open http://127.0.0.1:8765. Press *Run scripted demo*.
- **Terminal showcase:** python scripts/showcase.py prints the whole story in about a minute.
- **Tests:** pytest -q (183 pass).

4. Data and Clock Alignment

We use TraceBench, a public data set of traces from the Hadoop distributed file system under injected faults: 363 scenario folders, 81,777 traces, 3.0 GB. The folder name tells us the fault family, which gives the class label. Scenarios that inject several faults at once (20,010 traces) are held out as a probe and never trained on. The remaining 61,767 traces are split **by scenario**, so a fault seen in training never appears in testing.

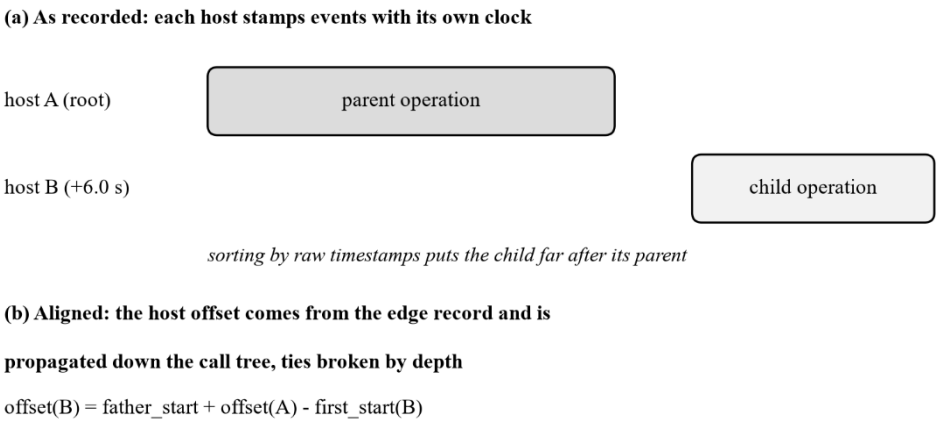


Figure 2: Each host records time on its own clock; the edge table lets us recover the offsets

In plain words. Two machines disagree about what time it is, so sorting events by their timestamps puts a child before its parent. The trace does record, for every child, what the parent's clock read when it spawned it. We use that to compute each machine's offset and walk down the call tree. All 81,777 traces align completely.

Table 4: The scenario-grouped split

Split	Scenarios	Traces	PASS	INFRA_FLAKE	DATA_FAULT	ENV_FAULT	REAL_BUG
Train	234	40,967	13,064	10,112	13,924	2,573	1,294
Validation	57	10,287	3,096	2,273	3,068	691	1,159
Test	57	10,513	2,964	2,319	3,068	832	1,330

5. The Triage Model

Each span becomes a token like *operation*|*status*, together with small embeddings of its host role, agent type and depth, and its duration. A bidirectional LSTM reads the sequence, and **attention pooling** combines all span states with learned weights instead of keeping only the last one. Nine trace-level features are added before the five-way output. The model has 125,430 parameters.

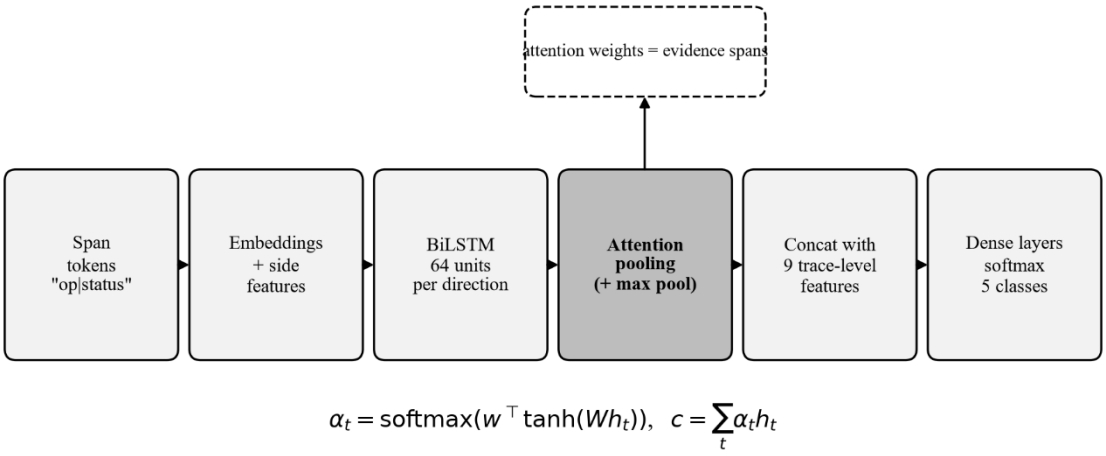


Figure 3: Architecture of the attention-pooled BiLSTM

Attention pooling in the model (src/features/triage_lstm.py)

```
scores = self.att(out).squeeze(-1)             # one score per span
scores = scores.masked_fill(~mask, -inf)        # ignore padding
w = torch.softmax(scores, dim=1)                # attention weights, sum to 1
attended = torch.bmm(w.unsqueeze(1), out)       # weighted sum of ALL span states
z = torch.cat([attended, pooled_max, self.feat(batch["feat"])], dim=-1)
logits = self.head(z)                           # 5 class scores
```

The attention weights come out with every prediction, so each decision can point to the spans that drove it. We treat these as a hint, not proof, because attention is known to be an imperfect explanation.

6. Comparison with Published Model Architectures

A fair question is how our model compares with the models used in the published work on log and trace failure analysis. Those papers evaluate on different data and tasks, so their reported numbers cannot be compared with ours directly. Instead we **re-implemented four of their architectures and trained them on our data**, with the same split, the same optimiser, class weights, seed and early stopping as our model. Nothing about the competitors was tuned to be weaker.

Table 5: The comparison models

Model	Based on	What it reads	How it classifies
DeepLog-style	Du et al., CCS 2017	operation tokens	2-layer LSTM, last hidden state (original is unsupervised next-key prediction; adapted to supervised)
LogRobust-style	Zhang et al., FSE 2019	operation tokens	attention-based BiLSTM (original uses semantic text vectors; here learned embeddings)
Span + timing LSTM	after Nedelkoski et al., CLOUD 2019	tokens + span duration	LSTM, last hidden state
TextCNN	Kim, EMNLP 2014	operation tokens	convolutions of width 2, 3, 4 with max-over-time pooling
DeepLog-style, our inputs	Du et al. backbone	exactly the inputs of the proposed model	2-layer LSTM, last hidden state (isolates architecture from inputs)
Proposed	this work	tokens + host, agent, depth, duration + 9 trace features	attention-pooled BiLSTM + max pool

Table 6: Results on the same 10,513 held-out test traces, sorted by macro-F1

Model (same data, split and recipe)	Based on	Params	Macro-F1	Excl. REAL_BUG	Accuracy
DeepLog-style LSTM given the same inputs as ours	Du et al. backbone, our inputs	88,437	0.7672	0.7090	0.7752
Proposed: attention BiLSTM + side features	this work	125,430	0.7464	0.6829	0.7690
Span + timing LSTM	after Nedelkoski et al., CLOUD 2019	35,749	0.6549	0.6483	0.6871
LogRobust-style: attention BiLSTM, tokens only	Zhang et al., FSE 2019	71,510	0.6115	0.6014	0.6213
TextCNN, tokens only	Kim, EMNLP 2014	32,981	0.6059	0.5980	0.6152
DeepLog-style: 2-layer LSTM, tokens only	Du et al., CCS 2017	66,965	0.4446	0.3930	0.5181

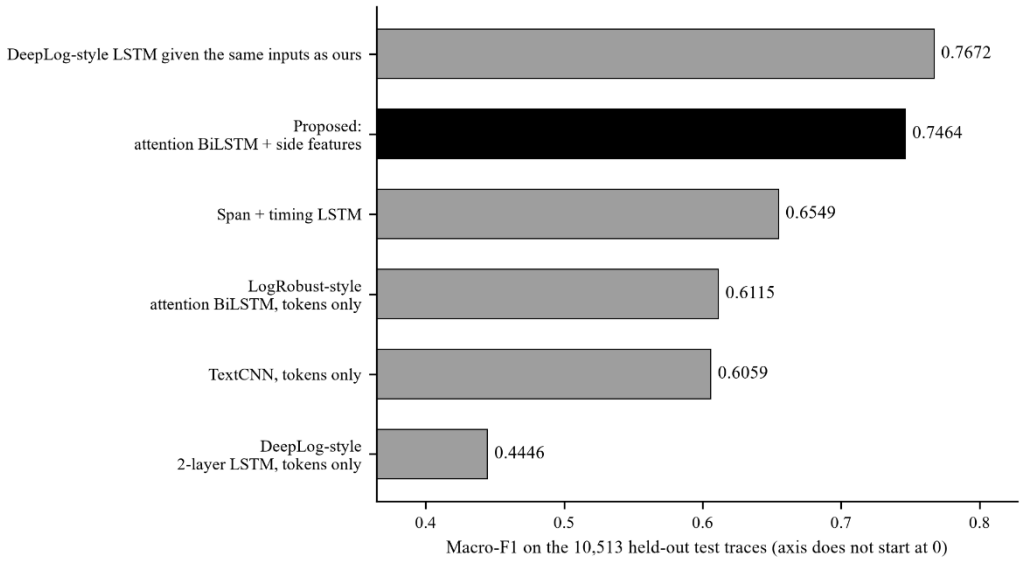


Figure 4: Macro-F1 of the proposed model and the re-implemented published architectures

The strongest competitor, DeepLog-style LSTM given the same inputs as ours, reaches a macro-F1 of 0.7672, which is 0.0208 above the proposed model (0.7464). Excluding REAL_BUG, whose score reflects trace length, the difference to the same model is -0.0261. The three models that see only operation tokens score 0.4446 to 0.6115, well below the models that also see timing or trace-level information. Adding span durations alone lifts a plain LSTM to 0.6549. Given exactly the same inputs as the proposed model, the DeepLog-style LSTM reaches 0.7672, a difference of -0.0208; this is the fairest measure of what the attention BiLSTM architecture itself contributes. All differences come from one training run per model with the same seed, so they should be read as comparable performance, not as a ranking with statistical weight.

How to read this honestly. (1) The competitors are re-implementations adapted to our task, not the original systems. (2) Three of them see only the operation tokens and one adds timing; the proposed model also sees host, depth, duration and trace-level features, so part of any advantage comes from richer input and not from the architecture. That is why the span-and-timing model and the DeepLog-style model given our exact inputs are included. (3) The order-free baseline B5 (0.7037) already shows that this data set rewards counts and scalars more than sequence structure. (4) One run per model, so small differences are within the noise we would expect between seeds.

7. What the Results Really Say

Table 7: Per-class results of the proposed model

Class	Precision	Recall	F1	Support
PASS	0.7935	0.6029	0.6852	2,964
INFRA_FLAKE	0.7238	0.7922	0.7564	2,319
DATA_FAULT	0.7267	0.9126	0.8091	3,068
ENV_FAULT	0.6111	0.3966	0.4810	832
REAL_BUG	1.0000	1.0000	1.0000	1,330

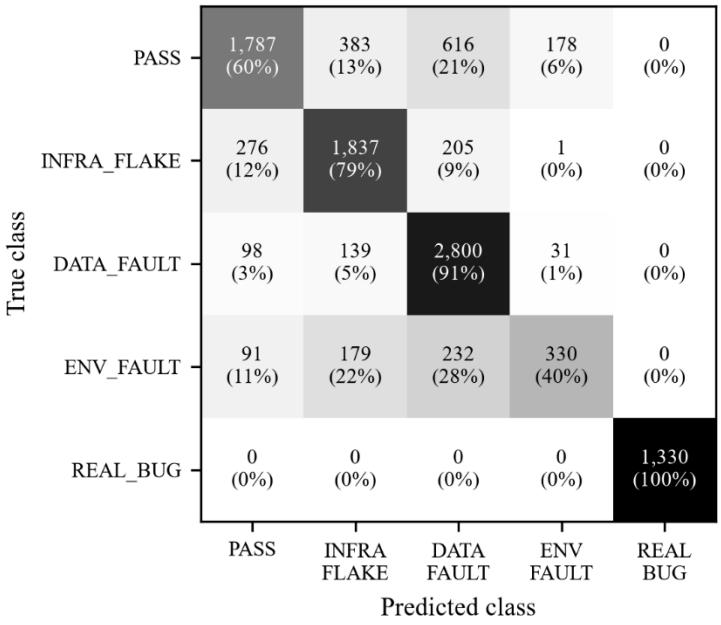


Figure 5: Confusion matrix on the 10,513 test traces

- **Architecture is not what wins here.** A plain DeepLog-style 2-layer LSTM given exactly our inputs scores 0.7672, slightly above our 0.7464 (it trained for 16 epochs against our 6). The advantage over published architectures comes from the richer inputs and the clock-aligned data, not from attention. We report this because it is what the experiment showed.
- **REAL_BUG's 1.000 is a workload effect.** The three REAL_BUG scenarios are short traces (median 7 spans on 2 hosts); every other class has 88–134 spans on 9–12 hosts. Trace length alone separates them, so we quote the figure excluding REAL_BUG (0.6829) as the honest headline.
- **Order carries little signal.** An order-free gradient-boosting baseline ties the BiLSTM on the other four classes, and shuffling tokens did not hurt (validation macro-F1 0.7552 shuffled vs 0.7429 ordered at epoch 3).
- **About 40% of healthy traces are flagged** (PASS recall 0.603): the weakest operational number. Only 139 traces (1.3%) were data faults that would have been wrongly retried, and no real defect was missed.
- **ENV_FAULT is the weak class** (F1 0.481). Softening the class weights traded recall for precision; which matters more is a policy decision.
- **Compound faults mostly fail:** 77.8% of the 20,010 held-out multi-fault traces were classified PASS.

8. Trust Engine: Earning and Losing Autonomy

The agent starts at T0 (observe only). It moves up a tier when the **Wilson lower bound** of its accuracy over fresh evidence clears the tier's threshold, and it drops a tier immediately if it classifies a real defect as harmless and acts on it without a human.

The promotion statistic (src/policy_engine/database.py)

```
def wilson_lower_bound(k, n, z=1.96):
    if n == 0:
        return 0.0
    p = k / n
    denom  = 1 + z*z / n
    centre = p + z*z / (2*n)
    spread = z * math.sqrt(p*(1-p)/n + z*z/(4*n*n))
    return max(0.0, (centre - spread) / denom)

# 17 correct out of 20 is 85% - but the lower bound is only 0.64
```

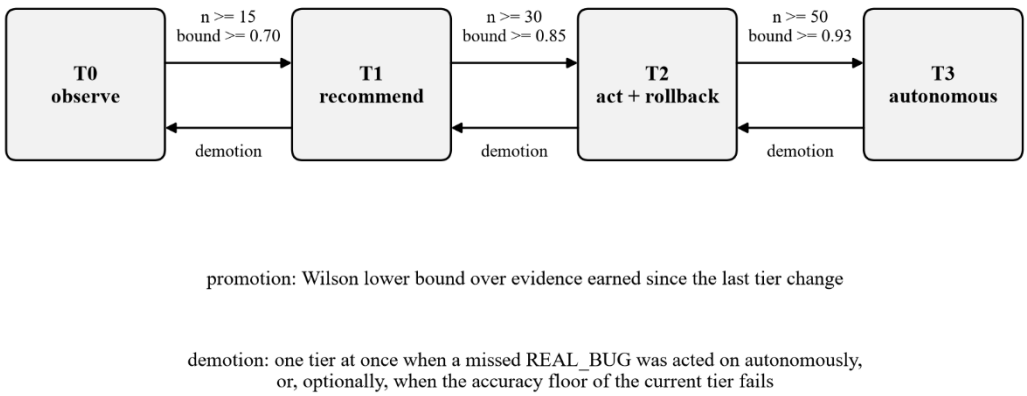


Figure 6: Trust tiers and the rules between them

Table 8: Thresholds per tier

Tier	Earned by (min. n, Wilson bound)	Retention floor	May execute
T0 observe	start	n/a	nothing
T1 recommend	15, 0.70	0.60	ESCALATE, BLOCK
T2 act with rollback	30, 0.85	0.75	adds ALLOW, RETRY, ROLLBACK
T3 autonomous	50, 0.93	0.83	adds QUARANTINE

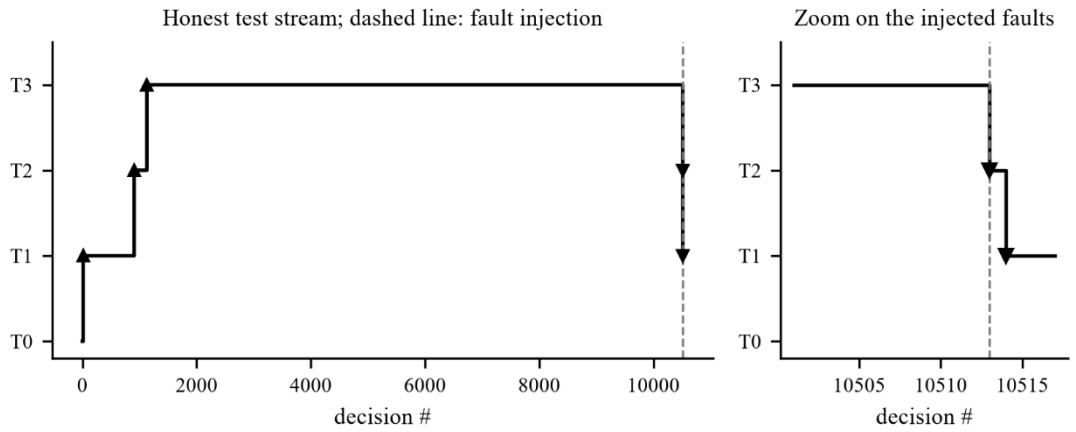


Figure 7: Replay over the 10,513 test decisions: promotions at decisions 14, 906 and 1,130, then demotions when faults are injected

A bug the live run found. After a demotion, the next correct decision re-ran the promotion test over the same history that had earned the tier, and bought it straight back: 53 tier changes in 50 decisions. The fix: only evidence earned since the last tier change counts.

9. The Release Gate and Its Supporting Modules

Gate precedence (src/policy_engine/gate.py, simplified)

```
if sec_v == "BLOCK":             recommended = BLOCK          # 1. security first
elif can_v == "ROLLBACK":         recommended = ROLLBACK       # 2. then the canary
elif not in_dist:                 recommended = REVIEW         # 3. abstain if out of
domain
elif triage is not None:          recommended = triage.action  # 4. otherwise the
triage decision
# the trust tier then decides whether the agent may EXECUTE it or only recommend it
```

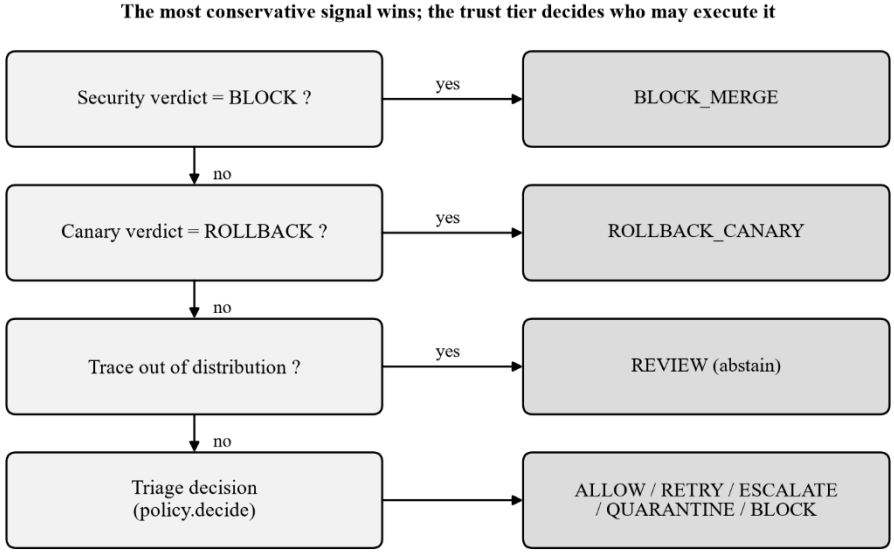


Figure 8: Precedence of the release gate

Table 9: The supporting modules and what was measured

Module	What it does	Measured result
Drift guard	Abstains if the trace's operations are unknown (OOV > 0.5) or its feature schema is not attested	Stock Jaeger exports: 20% accuracy and 90% called REAL_BUG without it; a foreign-app trace has OOV 1.0 and is routed to a human
Root-cause ranking	Scores spans against per-operation healthy latency; adds a bonus for errors	Top-3 vs proxy label: ENV_FAULT 94.6%, INFRA_FLAKE 73.4%, DATA_FAULT 47.6% (random 38.1%)
Canary check	Compares error rate and p95 latency of a new version with baseline using Wilson intervals	Healthy version promoted; slow database 16.7–20.9× p95 → rollback; flaky network 42.5% vs 0% errors → rollback
Security gate	Blocks only vulnerabilities the running service can reach	27 flagged, 0 reachable (12 pip, 6 tornado, 4 jupyterlab, 3 urllib3, 2 others)
Chaos harness	Injects six fault kinds plus a control into healthy traces	Silent regression: 0% classified correctly, 2 unsafe merges, trust T3 → T0; control flags 43% healthy traces
Actuator	Executes decisions: dry-run by default, argv-only commands, validated node names	Verified against a faked HTTP service; never run against real GitHub

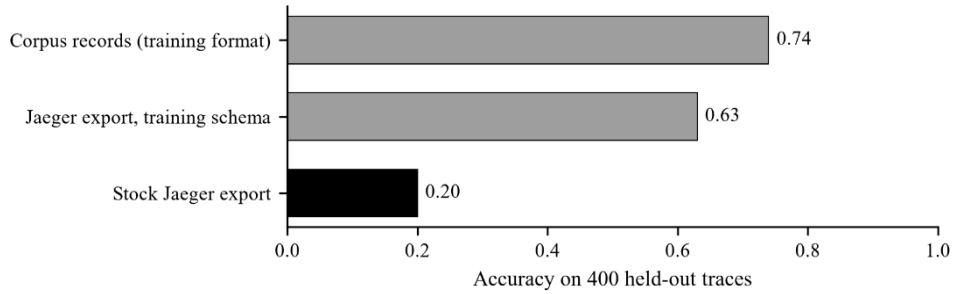


Figure 9: Accuracy of the model on corpus records versus two forms of Jaeger export

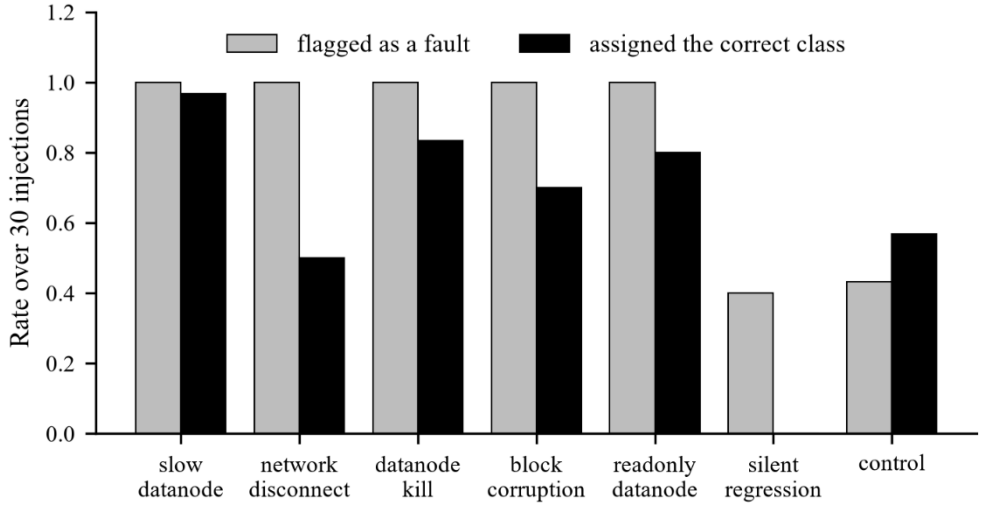


Figure 10: Chaos experiments: flagged rate and correct-class rate

10. The Live System and Dashboard

To show the whole loop on a running system, we built *minishop*: four services (frontend, orders, inventory, database) with real sleeps and retries, and a fault injector that knows exactly whether each fault took effect, so every trace has an exact label. A model trained on its 1,850 traces feeds the same pipeline. It scores 1.000 on its held-out scenarios, but so does a plain scalar baseline, so this demonstrates the pipeline and **is not an accuracy claim**.

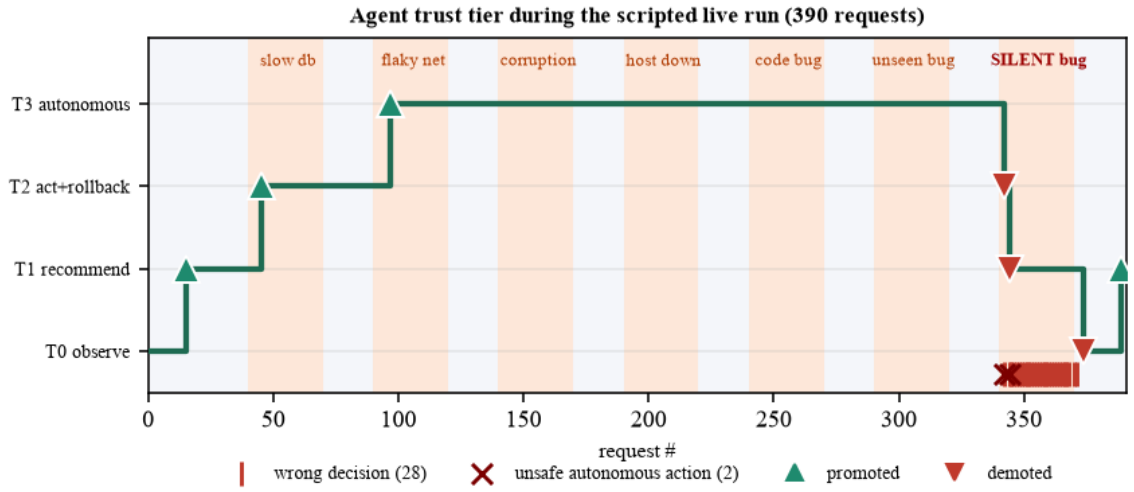


Figure 11: Trust tier during the scripted 390-request run

Table 10: Agent accuracy by phase of the scripted run

Phase	Requests	Correct	Rate
Steady (healthy)	180	179	99%
Slow database	30	30	100%
Flaky network	30	30	100%
Data corruption	30	30	100%
Host down	30	30	100%
Code bug	30	30	100%
Unseen bug	30	30	100%
Silent bug	30	3	10%
All phases	390	362	92.8%

The agent climbs to T3, is wrong on the silent bug (3 of 30), is demoted twice by the two unsafe merges and once more by the accuracy-retention check, and is promoted back to T1 only after fresh correct decisions. Repeated runs give 92.0–92.9% correct decisions.

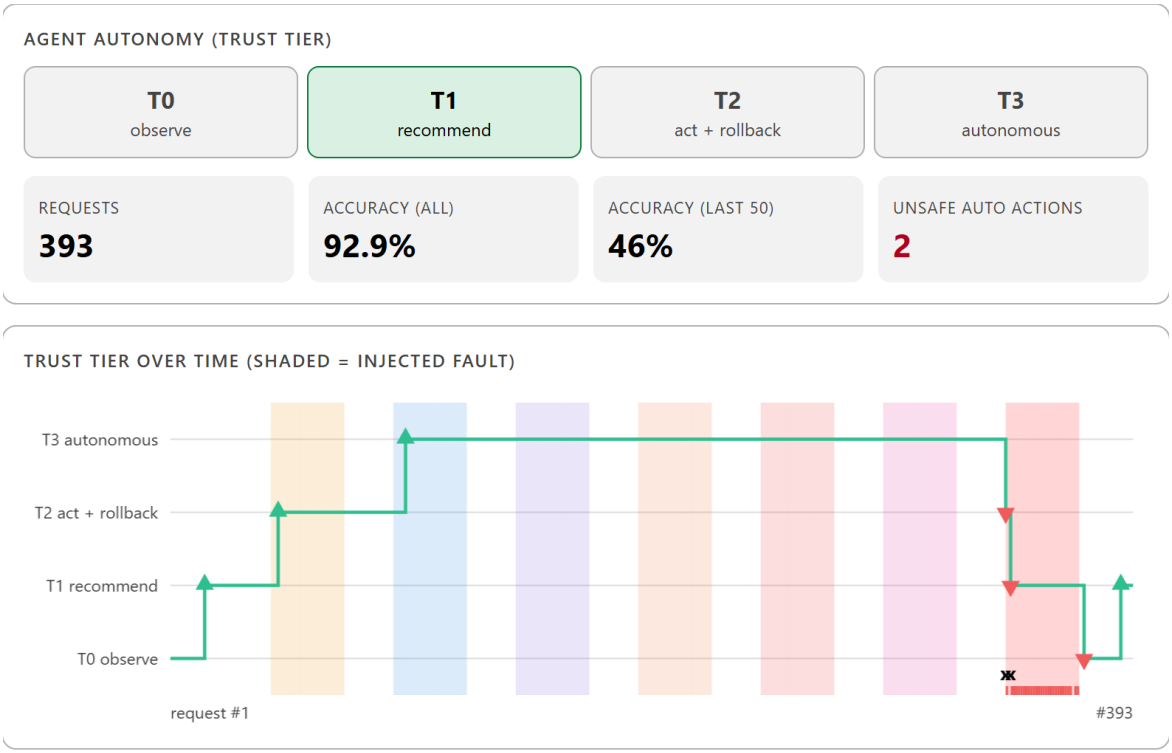


Figure 12: Dashboard: the live view at the end of the scripted run

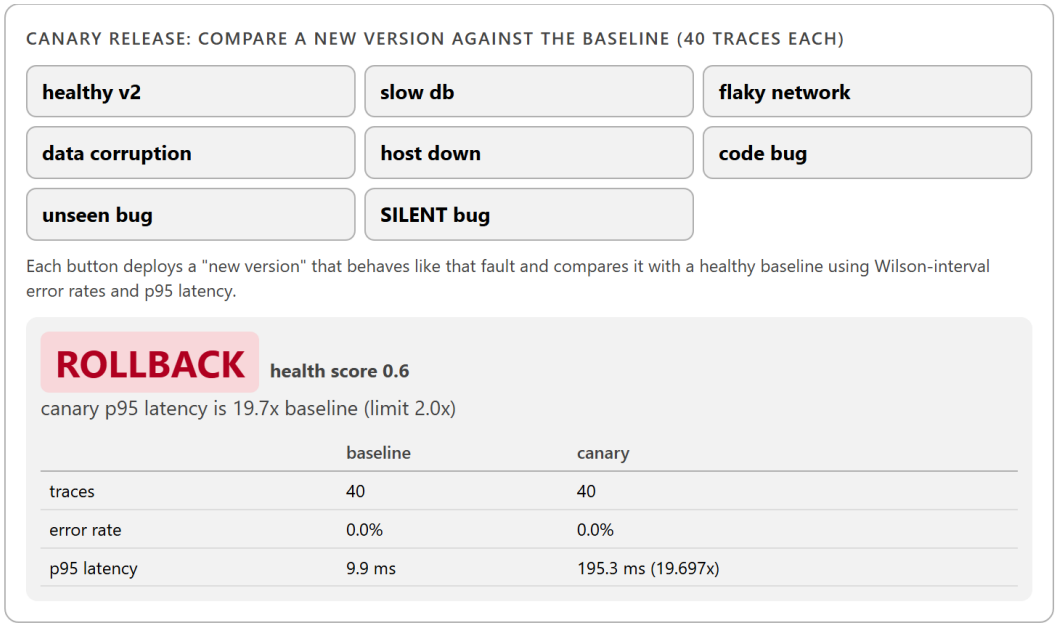


Figure 13: Dashboard: a canary release of a slow-database version is rolled back

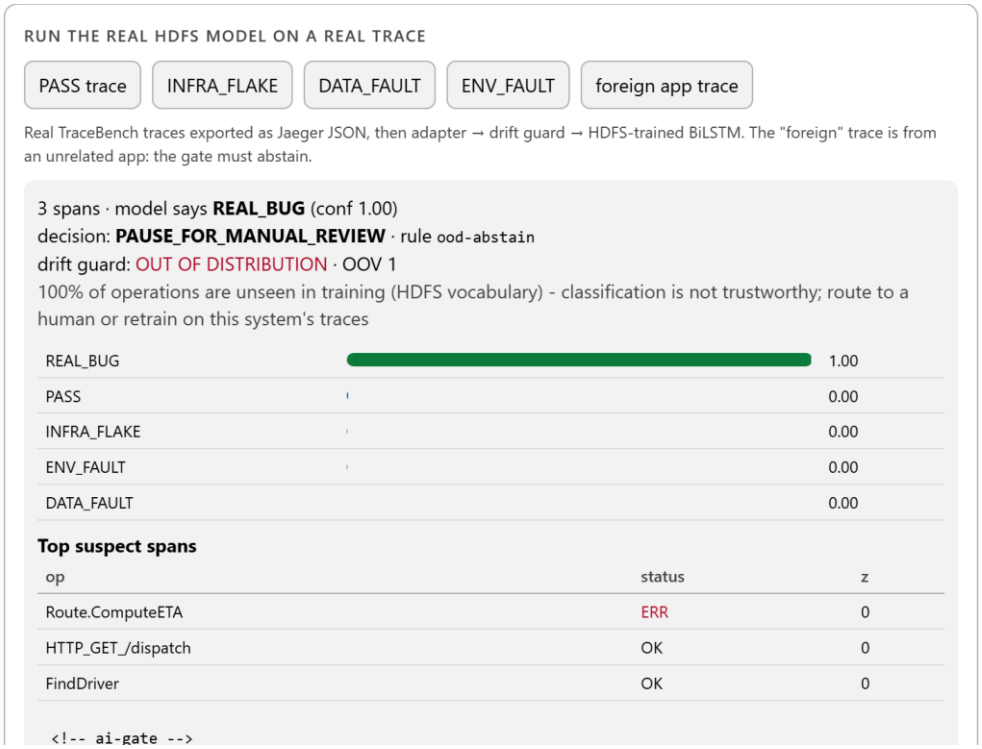


Figure 14: Dashboard: the HDFS model abstains on a trace from an unrelated application

11. Problems Found by Measuring

Table 11: Defects exposed by measurement and how they were corrected

Finding	Cause	Correction
Phase 1 accuracy of 88.18% frozen	Event-count vectors fed as token ids; a one-column rule scores 88.2%	Rebuilt from sequences; per-class metrics and baselines
Stock Jaeger exports judged with 20% accuracy	Missing trace-level features resemble the short REAL_BUG workload	Feature-schema condition in the drift guard
Live test accuracy 1.0 with classes missing	Per-class scenario assignment let PASS overwrite fault classes	Scenario assigned once by dominant fault class
53 tier changes in 50 decisions	A promotion re-used the evidence that had just failed to hold the tier	Only evidence since the last tier change counts
Token features shifted out of line	Operation names containing spaces	Names normalised; covered by a test
Two seconds lost on each local call (Windows)	localhost resolved via IPv6 first: 2,111 ms vs 26 ms	All scripts use 127.0.0.1
Evidence spans lost meaning	Attention regularisation made attention almost uniform	Both attempts rejected; spans reported as a hint

12. Limitations and Future Work

12.1 What we do not claim

- The model is valid only for HDFS traces in the training schema; the drift guard prevents a wrong answer outside it but cannot show the model is right inside it.
- The credible accuracy is the HDFS figure excluding REAL_BUG (0.6829); the live-system 1.000 is not a benchmark.

- Root-cause numbers are against a proxy label, not independently labelled root causes.
- The GitHub workflows, the Grafana/Prometheus stack and chaos on real infrastructure were written and structurally checked but never executed. Only Jaeger was run against a real server.
- The comparison uses re-implementations with one run per model.

12.2 Next steps

- Train and evaluate on a real microservice system with several independent fault scenarios per class.
- Reduce false alarms with a shape-matched REAL_BUG control and calibrated confidence.
- Move to a multi-label output for compound faults.
- Add signals outside the trace (test results, error budgets), because no trace-only model can see a silent regression.
- Run the unexecuted infrastructure and keep the AI gate advisory until the agent has earned T2 in deployment.

13. Demo Script (10 minutes)

- **1. Problem (1 min):** a red build looks the same whether it is a flake, a corruption, a dead host or a real bug.
- **2. Dashboard, Live tab (3 min):** press Run scripted demo; narrate the climb to T3; point at flaky network, host down, code bug being handled; then the silent bug and the demotions.
- **3. Release gate tab (1 min):** deploy the slow-database canary; it rolls back.
- **4. Security and Real data tabs (2 min):** 27 flagged, 0 reachable; the foreign-app trace abstains; show the baselines.
- **5. Comparison slide (2 min):** our model against the four published architectures on the same data.
- **6. Honest limits (1 min):** the three things we do not claim.